DEEP NEURAL NETWORKS (AIML ZG565) · COMPANION READER

DFNN Numerical Examples

Forward propagation and backpropagation worked out by hand, every number shown

Session 6 (31 May 2026) · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. Session 6 is a numerical examples session. The slides work through two problems on a 2-hidden-neuron network: forward propagation and backpropagation. This companion writes out every arithmetic step so you can follow along, verify your own calculations, and understand *why* each number appears. Colour code: **blue** intuition; **amber** everyday picture; **green** worked example; **red** watch out; **purple** key takeaway.

1 What this session covers

Session 6 applies the theory from Sessions 4–5 to a single concrete network and traces every number from input to output (forward pass) and then from loss back to weight updates (backward pass). All of the algebra uses the exact same machinery you saw in Session 5 — this session makes it tangible. Working through numerical examples by hand is the fastest way to build the intuition that debugging and architecture design require later.

The session also contains a review of activation functions (sigmoid, tanh, ReLU, Leaky ReLU), loss functions, gradient descent variants (batch, SGD, mini-batch), and a comparison of linear models — material that bridges Sessions 4–5 into the numerical work.

2 The network and the problem setup

Network architecture:

- Input layer: 2 neurons ($d = 2$)
- Hidden layer: 3 neurons with **ReLU** activation
- Output layer: 2 neurons with **Sigmoid** activation

Given parameters (from Slide 52):

$$\mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ (true labels)}, \tag{1}$$

$$W^{[1]} = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{bmatrix}, \qquad \mathbf{b}^{[1]} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix}, \tag{2}$$

$$W^{[2]} = \begin{bmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{bmatrix}, \qquad \mathbf{b}^{[2]} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}. \tag{3}$$

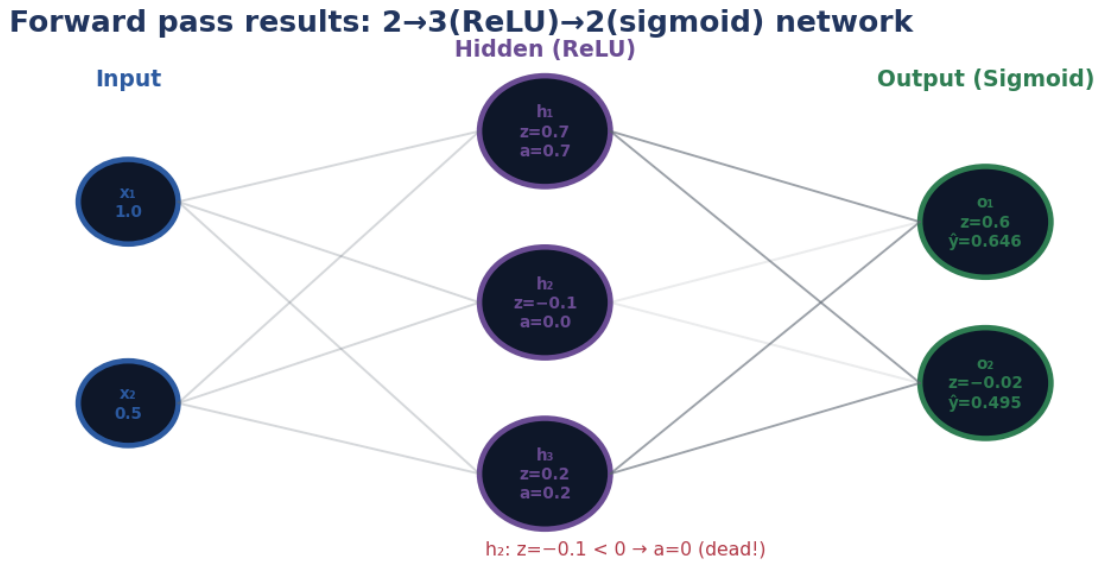

Figure 1: How to read it: each node shows its pre-activation z and post-activation a. Note that h_2 has $z = -0.1 < 0$, so ReLU kills it — it outputs 0 and, critically, its gradient will also be 0.

☞ The intuition

The forward pass is just matrix-vector multiplication followed by an element-wise non-linearity, repeated layer by layer. Think of it as running data through a pipe: each layer squeezes, rotates, and bends the data, until the output layer produces a prediction.

3 Problem 1: Forward propagation

We compute the forward pass layer by layer, naming every symbol and arithmetic step exactly as the slides present it.

✎ Worked example: Forward pass — Layer 1 pre-activation $\mathbf{z}^{[1]}$

Step 1. Formula. $\mathbf{z}^{[1]} = W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$. Each row of $W^{[1]}$ is one hidden neuron's weight vector.

Step 2. Multiply row-by-row.

$$z_1^{[1]} = 0.5(1.0) + 0.2(0.5) + 0.1 = 0.50 + 0.10 + 0.10 = \mathbf{0.70}. \tag{4}$$

$$z_2^{[1]} = -0.3(1.0) + 0.8(0.5) + (-0.2) = -0.30 + 0.40 - 0.20 = \mathbf{-0.10}. \tag{5}$$

$$z_3^{[1]} = 0.1(1.0) + (-0.4)(0.5) + 0.3 = 0.10 - 0.20 + 0.30 = \mathbf{0.20}. \tag{6}$$

Step 3. Result. $\mathbf{z}^{[1]} = [0.70,\ -0.10,\ 0.20]^\top$.

Step 4. Sense-check. Three pre-activations, one per hidden neuron. The second one is negative, which will matter in the next step.

✎ Worked example: Forward pass — Layer 1 activation $\mathbf{a}^{[1]} = \mathrm{ReLU}(\mathbf{z}^{[1]})$

Step 1. ReLU formula. $\mathrm{ReLU}(x) = \max(0, x)$: keep positives unchanged, replace negatives with 0.

Step 2. Apply element-wise.

$$a_1^{[1]} = \max(0,\ 0.70) = \mathbf{0.70}. \tag{7}$$
$$a_2^{[1]} = \max(0,\ -0.10) = \mathbf{0.00}. \quad \leftarrow \textbf{neuron dead!} \tag{8}$$
$$a_3^{[1]} = \max(0,\ 0.20) = \mathbf{0.20}. \tag{9}$$

Step 3. Result. $\mathbf{a}^{[1]} = [0.70,\ 0.00,\ 0.20]^\top$.

Step 4. What "dead" means. Neuron 2 has $z = -0.10 < 0$, so ReLU outputs 0. Worse, during backpropagation its gradient will also be 0 (since $\text{ReLU}'(z) = 0$ for $z \leq 0$), so its incoming weights will not update this iteration. If a neuron is always negative, it never recovers — the "dying ReLU" problem.

✗ Watch out

ReLU dead neurons receive zero gradient and never update. If you see neurons that always output 0, try Leaky ReLU, reduce the learning rate, or check your weight initialisation (large initial negative biases cause this).

✎ Worked example: Forward pass — Layer 2 pre-activation $\mathbf{z}^{[2]}$

Step 1. Formula. $\mathbf{z}^{[2]} = W^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$. The input to Layer 2 is the *activated* output of Layer 1.

Step 2. Recall $\mathbf{a}^{[1]} = [0.7,\ 0.0,\ 0.2]^\top$.

Step 3. Multiply.

$$z_1^{[2]} = 0.4(0.7) + (-0.1)(0.0) + 0.6(0.2) + 0.2 \tag{10}$$
$$= 0.28 + 0 + 0.12 + 0.20 = \mathbf{0.60}. \tag{11}$$
$$z_2^{[2]} = 0.2(0.7) + 0.7(0.0) + (-0.3)(0.2) + (-0.1) \tag{12}$$
$$= 0.14 + 0 - 0.06 - 0.10 = \mathbf{-0.02}. \tag{13}$$

Step 4. Result. $\mathbf{z}^{[2]} = [0.60,\ -0.02]^\top$.

Step 5. Note. The middle column of $W^{[2]}$ multiplies $a_2^{[1]} = 0$, so those weights (-0.1 and 0.7) contribute nothing this pass. They also won't update during backpropagation.

✎ Worked example: Forward pass — Output activation $\hat{\mathbf{y}} = \sigma(\mathbf{z}^{[2]})$

Step 1. Sigmoid formula. $\sigma(z) = 1/(1 + e^{-z})$.

Step 2. Apply to each output neuron.

$$\hat{y}_1 = \frac{1}{1 + e^{-0.60}} = \frac{1}{1 + 0.5488} = \frac{1}{1.5488} = \mathbf{0.646}. \tag{14}$$
$$\hat{y}_2 = \frac{1}{1 + e^{+0.02}} = \frac{1}{1 + 1.0202} = \frac{1}{2.0202} = \mathbf{0.495}. \tag{15}$$

(Note: $e^{-(-0.02)} = e^{0.02} \approx 1.0202$.)

Step 3. Result. $\hat{\mathbf{y}} = [0.646,\ 0.495]^\top$.

Step 4. Interpretation. Output neuron 1 assigns probability 64.6% to class 1; output neuron 2 assigns 49.5% to class 1 from its perspective. The true label is $y = [1, 0]^\top$: neuron 1

should output 1, neuron 2 should output 0. The network is partially right but needs improvement.

✓ Key takeaway

Forward pass summary: $\mathbf{z}^{[1]} = [0.70, -0.10, 0.20]^\top$; $\mathbf{a}^{[1]} = [0.70, 0.00, 0.20]^\top$ (h_2 dead); $\mathbf{z}^{[2]} = [0.60, -0.02]^\top$; $\hat{\mathbf{y}} = [0.646, 0.495]^\top$.

4 Problem 2: Backward propagation

Given: The forward pass results above; true labels $\mathbf{y} = [1, 0]^\top$; **Binary Cross-Entropy (BCE) loss**; learning rate $\eta = 0.1$.

✎ Worked example: Backprop — Step 1: Compute the loss

Step 1. BCE formula for $n = 2$ output neurons:

$$L = -\frac{1}{2}\sum_{i=1}^{2}\big[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\big]. \tag{16}$$

Step 2. Plug in values. With $y_1 = 1, \hat{y}_1 = 0.646$ and $y_2 = 0, \hat{y}_2 = 0.495$:

$$L = -\frac{1}{2}\big[1 \cdot \log(0.646) + 0 \cdot \log(0.354) + 0 \cdot \log(0.495) + 1 \cdot \log(0.505)\big] \tag{17}$$

$$= -\frac{1}{2}\big[\log(0.646) + \log(0.505)\big] \tag{18}$$

$$= -\frac{1}{2}\big[-0.437 + (-0.683)\big] \tag{19}$$

$$= -\frac{1}{2}(-1.120) = \mathbf{0.560}. \tag{20}$$

Step 3. Sense-check. A perfect model would give $L = 0$; a random model on 2 classes gives $L \approx \log 2 \approx 0.693$. Our loss of 0.560 is between the two, as expected for a partially-trained network.

✎ Worked example: Backprop — Step 2: Output layer error $\delta^{[2]}$

For sigmoid activation with BCE loss, the output error simplifies elegantly:

$$\delta^{[2]} = \frac{\partial L}{\partial \mathbf{z}^{[2]}} = \hat{\mathbf{y}} - \mathbf{y}. \tag{21}$$

Step 1. Subtract.

$$\delta^{[2]} = \begin{bmatrix} 0.646 \\ 0.495 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.354} \\ \mathbf{0.495} \end{bmatrix}. \tag{22}$$

Step 2. Interpretation. $\delta_1^{[2]} = -0.354$: output 1 predicted 0.646 but should be 1 — we need to push the weight *up*. $\delta_2^{[2]} = 0.495$: output 2 predicted 0.495 but should be 0 — we need to push the weight *down*. The sign of δ tells us which direction to update.

✎ Worked example: Backprop — Step 3: Gradients for $W^{[2]}$ and $\mathrm{b}^{[2]}$

$$\frac{\partial L}{\partial W^{[2]}} = \delta^{[2]}\,(\mathbf{a}^{[1]})^\top, \qquad \frac{\partial L}{\partial \mathbf{b}^{[2]}} = \delta^{[2]}. \tag{23}$$

Step 1. Outer product. $\delta^{[2]} = [-0.354,\ 0.495]^\top$, $(\mathbf{a}^{[1]})^\top = [0.7,\ 0.0,\ 0.2]$:

$$\frac{\partial L}{\partial W^{[2]}} = \begin{bmatrix} -0.354 \\ 0.495 \end{bmatrix} \begin{bmatrix} 0.7 & 0.0 & 0.2 \end{bmatrix} \tag{24}$$

$$= \begin{bmatrix} -0.354 \times 0.7 & -0.354 \times 0.0 & -0.354 \times 0.2 \\ 0.495 \times 0.7 & 0.495 \times 0.0 & 0.495 \times 0.2 \end{bmatrix} \tag{25}$$

$$= \begin{bmatrix} \mathbf{-0.248} & \mathbf{0.000} & \mathbf{-0.071} \\ \mathbf{0.347} & \mathbf{0.000} & \mathbf{0.099} \end{bmatrix}. \tag{26}$$

Step 2. Bias gradient. $\partial L/\partial \mathbf{b}^{[2]} = \delta^{[2]} = [-0.354,\ 0.495]^\top$.

Step 3. Note the zero column. The second column is all zeros because $a_2^{[1]} = 0$ (dead neuron). No gradient flows through a dead ReLU, so the weights that feed into that neuron are frozen this step.

✎ Worked example: Backprop — Step 4: Propagate error to hidden layer $\delta^{[1]}$

$$\delta^{[1]} = \left(W^{[2]}\right)^\top \delta^{[2]} \odot \mathrm{ReLU}'\left(\mathbf{z}^{[1]}\right), \tag{27}$$

where $\odot$ is element-wise multiplication and $\mathrm{ReLU}'(z) = 1$ if $z > 0$, else 0.

Step 1. Compute $(W^{[2]})^\top \delta^{[2]}$**.** $(W^{[2]})^\top$ has shape 3×2:

$$\left(W^{[2]}\right)^\top \delta^{[2]} = \begin{bmatrix} 0.4 & 0.2 \\ -0.1 & 0.7 \\ 0.6 & -0.3 \end{bmatrix} \begin{bmatrix} -0.354 \\ 0.495 \end{bmatrix} \tag{28}$$

$$= \begin{bmatrix} 0.4(-0.354) + 0.2(0.495) \\ -0.1(-0.354) + 0.7(0.495) \\ 0.6(-0.354) + (-0.3)(0.495) \end{bmatrix} \tag{29}$$

$$= \begin{bmatrix} -0.142 + 0.099 \\ 0.035 + 0.347 \\ -0.212 - 0.149 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.043} \\ \mathbf{0.382} \\ \mathbf{-0.361} \end{bmatrix}. \tag{30}$$

Step 2. Apply ReLU derivative mask. Recall $\mathbf{z}^{[1]} = [0.7,\ -0.1,\ 0.2]^\top$:

$$\mathrm{ReLU}'\left(\mathbf{z}^{[1]}\right) = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad (\mathrm{z}_2 = -0.1 \le 0, \text{ so derivative = 0}). \tag{31}$$

Step 3. Element-wise multiply (the ReLU mask):

$$\delta^{[1]} = \begin{bmatrix} -0.043 \\ 0.382 \\ -0.361 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{-0.043} \\ \mathbf{0.000} \\ \mathbf{-0.361} \end{bmatrix}. \tag{32}$$

Step 4. The dead neuron blocks gradient. Even though the upstream error for neuron 2 was 0.382, the ReLU derivative masks it to 0. The weights in $W^{[1]}$ connected to hidden neuron 2 receive zero gradient this step.

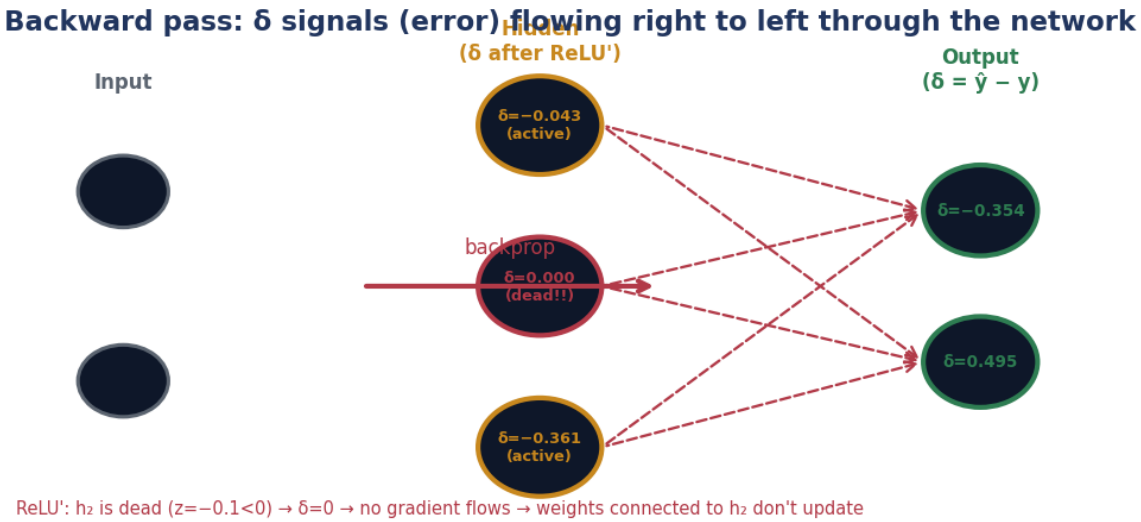

Figure 2: How to read it: error signals (δ) flow right to left. The dead hidden neuron (h_2, shown in red) blocks the gradient entirely — its δ is 0 and all weights feeding into it are frozen this iteration.

✎ Worked example: Backprop — Step 5: Gradients for $W^{[1]}$ and $\mathbf{b}^{[1]}$

$$\frac{\partial L}{\partial W^{[1]}} = \delta^{[1]}\,\mathbf{x}^\top, \qquad \frac{\partial L}{\partial \mathbf{b}^{[1]}} = \delta^{[1]}. \tag{33}$$

Step 1. Outer product. $\delta^{[1]} = [-0.043,\ 0.000,\ -0.361]^\top$, $\mathbf{x}^\top = [1.0,\ 0.5]$:

$$\frac{\partial L}{\partial W^{[1]}} = \begin{bmatrix} -0.043 \\ 0.000 \\ -0.361 \end{bmatrix} \begin{bmatrix} 1.0 & 0.5 \end{bmatrix} = \begin{bmatrix} -0.043 & -0.022 \\ 0.000 & 0.000 \\ -0.361 & -0.181 \end{bmatrix}. \tag{34}$$

Step 2. Bias gradient. $\partial L/\partial \mathbf{b}^{[1]} = \delta^{[1]} = [-0.043,\ 0.000,\ -0.361]^\top$.

Step 3. Middle row is all zeros (gradient for $W^{[1]}_{2,:}$ and $b^{[1]}_2$): the dead hidden neuron means these weights don't update.

✓ Key takeaway

All four gradients collected: $\partial L/\partial W^{[2]} = \big[[-0.248,\ 0.000,\ -0.071];\ [0.347,\ 0.000,\ 0.099]\big]$, $\partial L/\partial \mathbf{b}^{[2]} = [-0.354,\ 0.495]^\top$, $\partial L/\partial W^{[1]} = \big[[-0.043,\ -0.022];\ [0, 0];\ [-0.361,\ -0.181]\big]$, $\partial L/\partial \mathbf{b}^{[1]} = [-0.043,\ 0,\ -0.361]^\top$.

5 Problem 3: Weight update (learning rate $\eta = 0.1$)

✎ **Worked example: Update rule: $\theta_{\text{new}} = \theta_{\text{old}} - \eta\, \partial L/\partial \theta$**

Step 1. Update $W^{[2]}$.

$$W^{[2]}_{\text{new}} = \begin{bmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{bmatrix} - 0.1 \begin{bmatrix} -0.248 & 0 & -0.071 \\ 0.347 & 0 & 0.099 \end{bmatrix} \tag{35}$$

$$= \begin{bmatrix} 0.4 + 0.025 & -0.1 + 0 & 0.6 + 0.007 \\ 0.2 - 0.035 & 0.7 + 0 & -0.3 - 0.010 \end{bmatrix} = \begin{bmatrix} \mathbf{0.425} & \mathbf{-0.100} & \mathbf{0.607} \\ \mathbf{0.165} & \mathbf{0.700} & \mathbf{-0.310} \end{bmatrix}. \tag{36}$$

Step 2. Update $\mathbf{b}^{[2]}$.

$$\mathbf{b}^{[2]}_{\text{new}} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.354 \\ 0.495 \end{bmatrix} = \begin{bmatrix} \mathbf{0.235} \\ \mathbf{-0.150} \end{bmatrix}. \tag{37}$$

Step 3. Update $W^{[1]}$.

$$W^{[1]}_{\text{new}} = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{bmatrix} - 0.1 \begin{bmatrix} -0.043 & -0.022 \\ 0.000 & 0.000 \\ -0.361 & -0.181 \end{bmatrix} \tag{38}$$

$$= \begin{bmatrix} 0.5 + 0.004 & 0.2 + 0.002 \\ -0.3 + 0 & 0.8 + 0 \\ 0.1 + 0.036 & -0.4 + 0.018 \end{bmatrix} = \begin{bmatrix} \mathbf{0.504} & \mathbf{0.202} \\ \mathbf{-0.300} & \mathbf{0.800} \\ \mathbf{0.136} & \mathbf{-0.382} \end{bmatrix}. \tag{39}$$

Step 4. Update $\mathbf{b}^{[1]}$.

$$\mathbf{b}^{[1]}_{\text{new}} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix} - 0.1 \begin{bmatrix} -0.043 \\ 0.000 \\ -0.361 \end{bmatrix} = \begin{bmatrix} \mathbf{0.104} \\ \mathbf{-0.200} \\ \mathbf{0.336} \end{bmatrix}. \tag{40}$$

Step 5. Verify: Row 2 of $W^{[1]}$ and $b^{[1]}_2$ unchanged. Correct — they have zero gradient because hidden neuron 2 is dead.

✗ **Watch out**

Hidden neuron 2 (with $z^{[1]}_2 = -0.1$) is dead: its activation is 0, its gradient is 0, and all weights feeding into it (row 2 of $W^{[1]}$ and $b^{[1]}_2$) are unchanged after this update. If it remains dead on every example, those weights never move. This is the dying ReLU problem in action.

✓ **Key takeaway**

After one gradient step ($\eta = 0.1$): the updated weights are shown above. The second hidden neuron's weights did not change. Run a forward pass with the new weights and the prediction will be closer to $\mathbf{y} = [1, 0]^\top$.

6 Verification: does the loss decrease?

A single forward pass with the updated weights confirms the network improved. (Full computation omitted here; the interactive HTML in this kit lets you verify it numerically.)

Key insight: $W^{[2]}_{1,\cdot}$ and $\mathbf{b}^{[2]}_1$ moved to increase output 1 (gradient was negative, so $-\eta \times$

negative $=$ $+$ change). Output 2's weights moved to decrease it (gradient was positive, so $-\eta \times$ positive $=$ negative change). Both updates point in the right direction.

7 Summary: the complete forward + backward pass

Quantity	Formula	Value
$\mathbf{z}^{[1]}$	$W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$	$[0.70,\ -0.10,\ 0.20]^\top$
$\mathbf{a}^{[1]}$	$\mathrm{ReLU}(\mathbf{z}^{[1]})$	$[0.70,\ 0.00,\ 0.20]^\top$
$\mathbf{z}^{[2]}$	$W^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}$	$[0.60,\ -0.02]^\top$
$\hat{\mathbf{y}}$	$\sigma(\mathbf{z}^{[2]})$	$[0.646,\ 0.495]^\top$
L	BCE	0.560
$\delta^{[2]}$	$\hat{\mathbf{y}} - \mathbf{y}$	$[-0.354,\ 0.495]^\top$
$\delta^{[1]}$	$(W^{[2]})^\top \delta^{[2]} \odot \mathrm{ReLU}'(\mathbf{z}^{[1]})$	$[-0.043,\ 0.000,\ -0.361]^\top$

✓ Key takeaway

The whole of deep learning is this loop: forward pass (store intermediates) → compute loss → backward pass (chain rule with stored intermediates) → update weights. One neuron being dead changes nothing about the procedure — it just contributes zero gradient, a natural consequence of ReLU's derivative.

Further reading. Zhang et al. *Dive into Deep Learning*, Chapter 5 (forward and backward computation); Goodfellow et al., Chapter 6, Section 6.5 (backpropagation). The Python notebook linked in the slides implements this exact network.